

COMP 1005B/1405B

Winter 2022 - Tutorial 4

Objectives

- Practice writing code using **looping** control structures
- Practice using **conditional/branching** control structures
- Practice using basic Python **input/output** operations

Expectations

To receive full grades for this tutorial, you must complete problems 1 - 4. Problems 5 and 6 are for practice. If you do not have time to complete the problems, you should attempt them after the tutorial for your own benefit.

Submit

For this tutorial, your group will submit a single zip file called **YourGroupName_T4.zip** to the appropriate channel in Discord, where **YourGroupName** should be replaced by *your group name*. The zip file should contain the attendance record of your group members and the solutions to all the required problems. If any group member is absent during a tutorial session and does not participate in any group work during that entire week, your group must report it.

Ensure that your tutorial grade is entered on Brightspace at the end of each tutorial week. If you don't see your previous grades, contact the Tutorial TAs immediately. You have no more than 1 week to report a missing tutorial grade.

Warm-up

Problem 1

Write a program called **p1.py** that asks the user for a number from 1 to 9 and displays a well-formatted multiplication table of all numbers up to the user's input. Your solution should not hard-code the entire table in print statements, but rather, use nested loops to print each number based on the user's input.

Hints:

- Use nested for loops, the inner for loop can work with the rows (values on the horizontal lines) and the outer for loop can work with the columns (values on the vertical lines).
- Use tab ("\t") to format the columns of the table.

Sample Outputs:

Enter a number (1-9): 5

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25

Enter a number (1-9): 9

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

Hangman Game

Problem 2

Write a program called **p2.py** that determines the length of a user inputted string.

Note: this program should NOT use the built-in `len()` function.

Sample output:

```
Enter a word: copybara
The length of copybara is 8
```

This problem will help you practice using for loops. Remember the `for` and `in` keywords. You can access each letter of a string by using a for loop as follows.

```
for letter in "cat":
    print(letter)
```

will print the following.

```
c
a
t
```

Problem 3

Write a program called **p3.py** that does the following:

1. prompts the user to enter a word
2. using that word, first prints out the word where all characters are replaced by “_”s (there will be an example to better illustrate this below).
3. the program will continue to print the word revealing one character at a time. i.e., the second line printed should print the first character followed by “_”s representing the rest of the word.
4. The program should end after printing the entire word once.

Sample output:

Enter a word: kangaroo

```
_____  
k_____  
ka_____  
kan_____  
kang_____  
kanga_____  
kangar_____  
kangaro_____  
kangaroo
```

Note: you will have to use nested loops (a loop inside another loop) in this problem. For example,

```
while/for condition1:  
    statements  
    while/for condition2:  
        more statements
```

Note: some things that might be helpful to think about when tackling this problem:

- the relationship between the number of lines / variations printed and the length of the word
- the relationship between the line number, the length of the word, and the number of underscores printed.
- the relationship between the line number and the number of letters that should be printed (not as underscores)

This problem will help you practice using loops, nested loops, and pattern analysis.

Note: the python `print()` function automatically adds a newline character to the end of each print statement (i.e., it prints and moves to the next line). In order to prevent / change this, you add `end = ""`.

Example removing the automatic newline:

```
print("some text", end = "")  
# in this case nothing is added to the end,  
# but a space could be added using end = " "
```

Problem 4

Write a program called **p4.py** that allows a user to guess if certain letters are in a word. The program should do the following:

1. Ask for user to input a letter to guess.
2. Check if the input is valid. Valid input is a single letter or 'quit', both case insensitive
3. Output how many times that letter appears in the word (see example below).
4. The program should continue functionality 1-3 infinitely until the user enters 'quit'. When the user enters 'quit', the word should be displayed, and the program should end.

Example:

```
Enter a letter to guess: A
there are 1 a's
Enter a letter to guess: 2
Invalid input
Enter a letter to guess: e
there are 0 e's
Enter a letter to guess: quit
The word was 'wombat'
Goodbye
```

Note: feel free to select any word but be sure to test your program using many different words.

DNA Project

Problem 5

Write a program called **p5.py** that continuously prompts the user to input a DNA strand and checks if it is valid. In order for the strand to be valid, it must only be made up of the following characters: 'A', 'T', 'C', and 'G' (in any combination; case insensitive). The program should terminate when the user enters 'q' or 'Q'.

Example:

```
Enter a DNA strand (or 'q' to quit): AATCGCT
Valid
Enter a DNA strand (or 'q' to quit): GGCBT
Invalid
Enter a DNA strand (or 'q' to quit): aAcGGT
Valid
Enter a DNA strand (or 'q' to quit): q
Goodbye
```

Note: this will help you practice / understand while loops, infinite loops, iterating (using for loops), and input checking.

Problem 6

Write a program called **p6.py** that determines the hamming distance of two DNA strands. Your program should do the following:

1. prompt the user for 2 DNA strands
2. Making use of problem 4, check if the strands are valid. If the inputted strands are of different lengths, this is considered invalid input. If one or both of the strands are invalid, the program ends.
3. Iterate over the two DNA strands and sum the number of differences.
4. Output the hamming distance.

Example:

```
Enter a DNA strand: AATG
Enter a second DNA strand: AATC
The hamming distance of AATG and AATC is 1
```

```
Enter a DNA strand: AATG
Enter a second DNA strand: T
Invalid input
```

```
Enter a DNA strand: ABC
Enter a second DNA strand: AATG
Invalid input
```

```
Enter a DNA strand: AATG
Enter a second DNA strand: TTAC
The hamming distance of AATG and TTAC is 4
```

Note: the hamming distance is the number of differences / differing characters in two strings.

Examples:

- 'AATG' and 'AATC' have a hamming distance of 1 because only the last character of the two strands differs.
- 'AATG' and 'ATAG' have a hamming distance of 2.
- 'AATG' and 'TTAC' have a hamming distance of 4.
 - Note, here the hamming distance is the length, and the two strands are complements.

Challenge: try and minimize the number of loops used.